

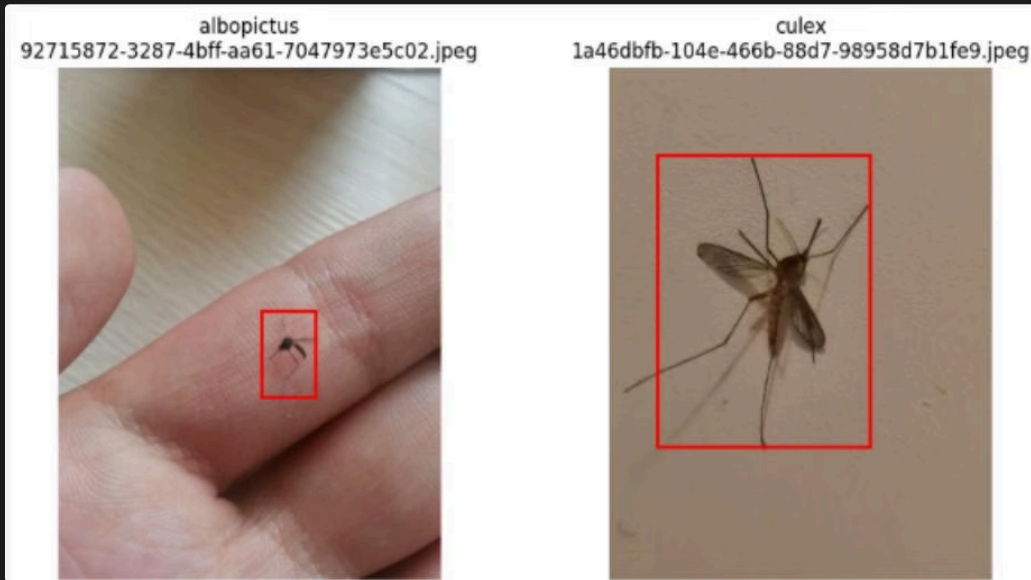
PROYECTO 2

Clasificación de Mosquitos

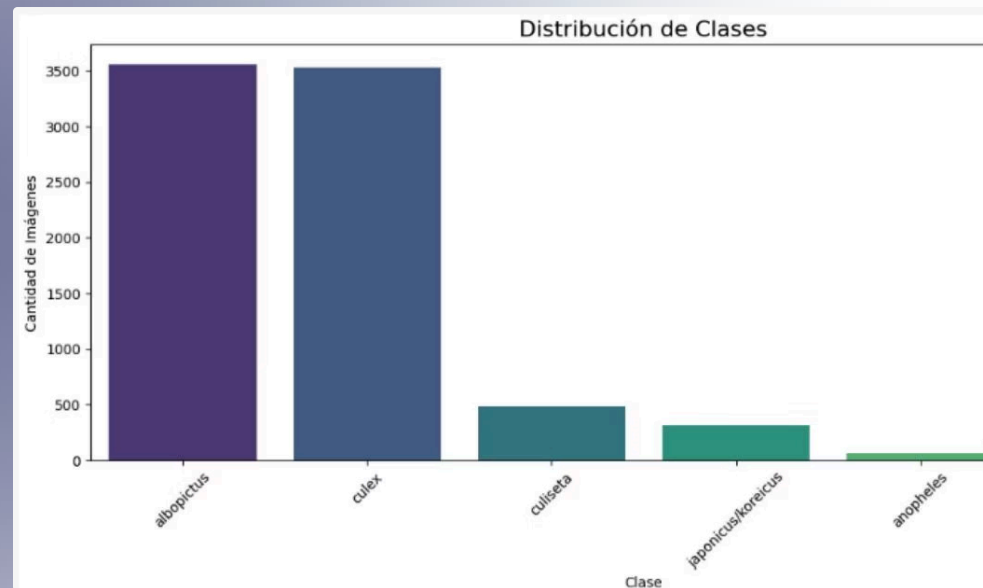


Introducción al Desafío

Este proyecto aborda el "Mosquito Alert Challenge", un desafío de clasificación de mosquitos. El objetivo es identificar y clasificar mosquitos en imágenes usando visión por computadora.



La tarea y objetivo principal es detectar la región del mosquito (bounding boxes) y clasificarlo en diferentes especies.



Análisis Exploratorio de Datos (EDA)

Durante el EDA, identificamos un desequilibrio significativo en las clases, lo que afecta la representación y la variación posicional de las especies minoritarias.

Desbalance de Clases

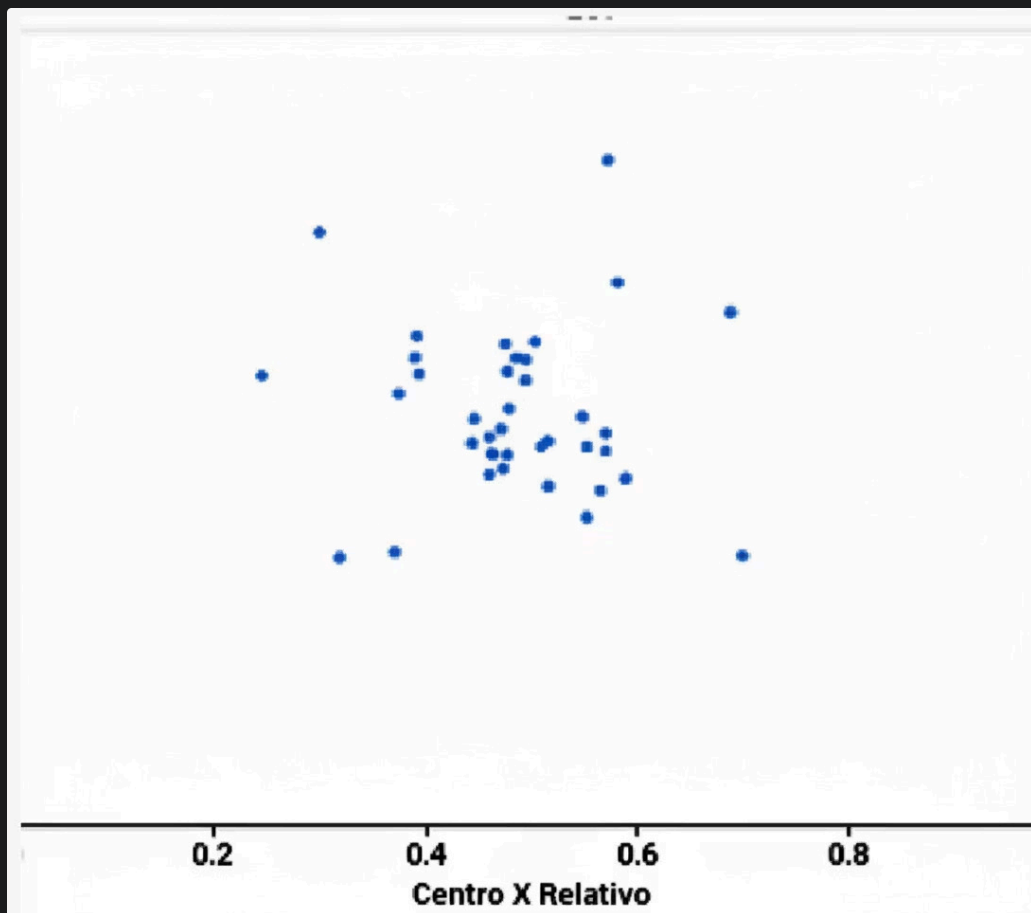
Fuerte desequilibrio en la distribución de las especies de mosquitos en el dataset.

Variación Posicional

Las clases minoritarias carecen de variación en las posiciones de los mosquitos en las imágenes.

Hallazgos Clave del Dataset

El dataset presenta gran variabilidad en tamaño, resolución y relación de aspecto de las imágenes, con casos atípicos extremos.



- Bounding boxes centrados y pequeños, pero con excepciones donde el mosquito ocupa gran parte de la foto.
- El tamaño de la imagen no se correlaciona con el del bounding box; no hay patrones claros por especie en la posición.
- Especies como *anopheles* y *aegypti* tienen menos variabilidad en resolución, lo que puede sesgar el aprendizaje.
- Desbalance notable: *albopictus* y *culex* representan el 88% de las observaciones.

Pipeline de Limpieza de Datos durante EDA

Durante la fase de Análisis Exploratorio de Datos (EDA), se implementaron las siguientes etapas de limpieza para refinar el dataset y mejorar la calidad de los datos:

- **Eliminación de imágenes con recortes excesivos:** Se descartaron imágenes donde los recortes (crops) superaban el 80% del área, basándose en estándares de visión artificial para asegurar la integridad de la escena.
- **Corrección de bounding boxes fuera de límites:** Se eliminaron o ajustaron los bounding boxes que excedían las dimensiones de la imagen, garantizando que todas las anotaciones estuvieran dentro de los límites válidos.
- **Filtrado de objetos pequeños:** Se removieron los objetos cuyo tamaño era de 10 píxeles o menos, lo que ayuda a reducir el ruido y la detección de elementos irrelevantes para el entrenamiento del modelo.

Pipeline de Preprocesamiento

Implementamos un pipeline robusto para preparar los datos, asegurando uniformidad y mitigando el desequilibrio de clases, utilizando pytorch

01

Transformación de Imágenes

Uniformizamos los tamaños de las imágenes en todo el dataset.

02

Data Augmentation

Aplicamos rotaciones y transformaciones de posición para mejorar la generalización del modelo.

03

WeightedRandomSampler

Utilizamos un muestreo ponderado para proteger las clases minoritarias del desequilibrio.



Modelos de Detección

Evaluamos tres modelos de detección principales, cada uno con sus propias fortalezas y debilidades.

1

Faster R-CNN

Modelo de dos etapas, alta precisión y fiabilidad, ideal para aplicaciones científicas.

Es un detector de dos etapas que utiliza una Red de Propuestas de Región (RPN) para identificar regiones de interés antes de clasificarlas y refinar sus cuadros delimitadores, lo que contribuye a su alta precisión.

2

RetinaNet

Detector de una sola etapa que utiliza **FPN** y la función de pérdida **Focal Loss**, logrando un gran equilibrio entre precisión y eficiencia.

Es un detector **basado en anchors**, y la Focal Loss reduce el desequilibrio entre ejemplos negativos (fondo) y positivos, permitiendo una detección más estable y precisa en escenarios con muchas regiones sin objetos.

3

YOLOv8

Detección en una sola pasada, rápido y liviano, ideal para tiempo real.

Utiliza una arquitectura moderna **anchor-free** con extracción de características profunda y una cabeza de detección eficiente para predecir **cajas y clases en un solo paso**, logrando un excelente equilibrio entre velocidad y precisión.

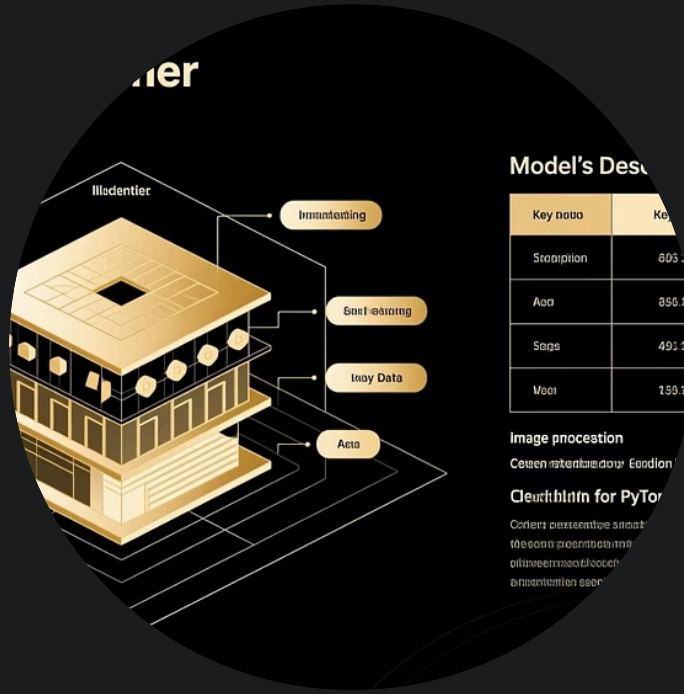
Modelos de Clasificación

Para la clasificación, consideramos varios modelos, incluyendo arquitecturas avanzadas y enfoques integrados.



ConvNext

Es una ConvNet modernizada que incorpora principios de diseño de Vision Transformers, utilizando convoluciones de profundidad, normalización de capas y activaciones GELU para un rendimiento mejorado manteniendo la eficiencia de las ConvNet.



ViT Base

Vision Transformer con parche de 16x16.

Este Vision Transformer trata las imágenes como secuencias de parches (16x16), utilizando mecanismos de autoatención para capturar dependencias globales en toda la imagen, revolucionando la visión por computadora con la arquitectura Transformer.



YOLOv8-cls

Modelo YOLO para clasificación

Es un modelo que comparte arquitectura con su contraparte de detección, pero la cabeza funciona para clasificación. Forma parte del entorno YOLO, el cual se encarga en su mayoría del proceso de entrenamiento.

Desafíos del Proyecto



Limitaciones de Cómputo

Los modelos de vision por computadora requieren de un poder computacional muy alto, por lo tanto nos vimos limitados por restricciones de tiempo y computo.



Familiaridad con Visión por Computadora

Los miembros del equipo tuvieron que familiarizarse con conceptos relacionados a vision por computadora y diferentes tecnologías.



Estimación de 'Justicia' entre Modelos

Dificultad para definir y medir criterios de justicia o equidad en el rendimiento de diferentes modelos, especialmente utilizando diferentes frameworks.



Aprendizaje de Pipelines

Dominar dos frameworks distintos (YOLO y TorchVision) para diferentes tareas presenta una complejidad adicional en el desarrollo del proyecto.